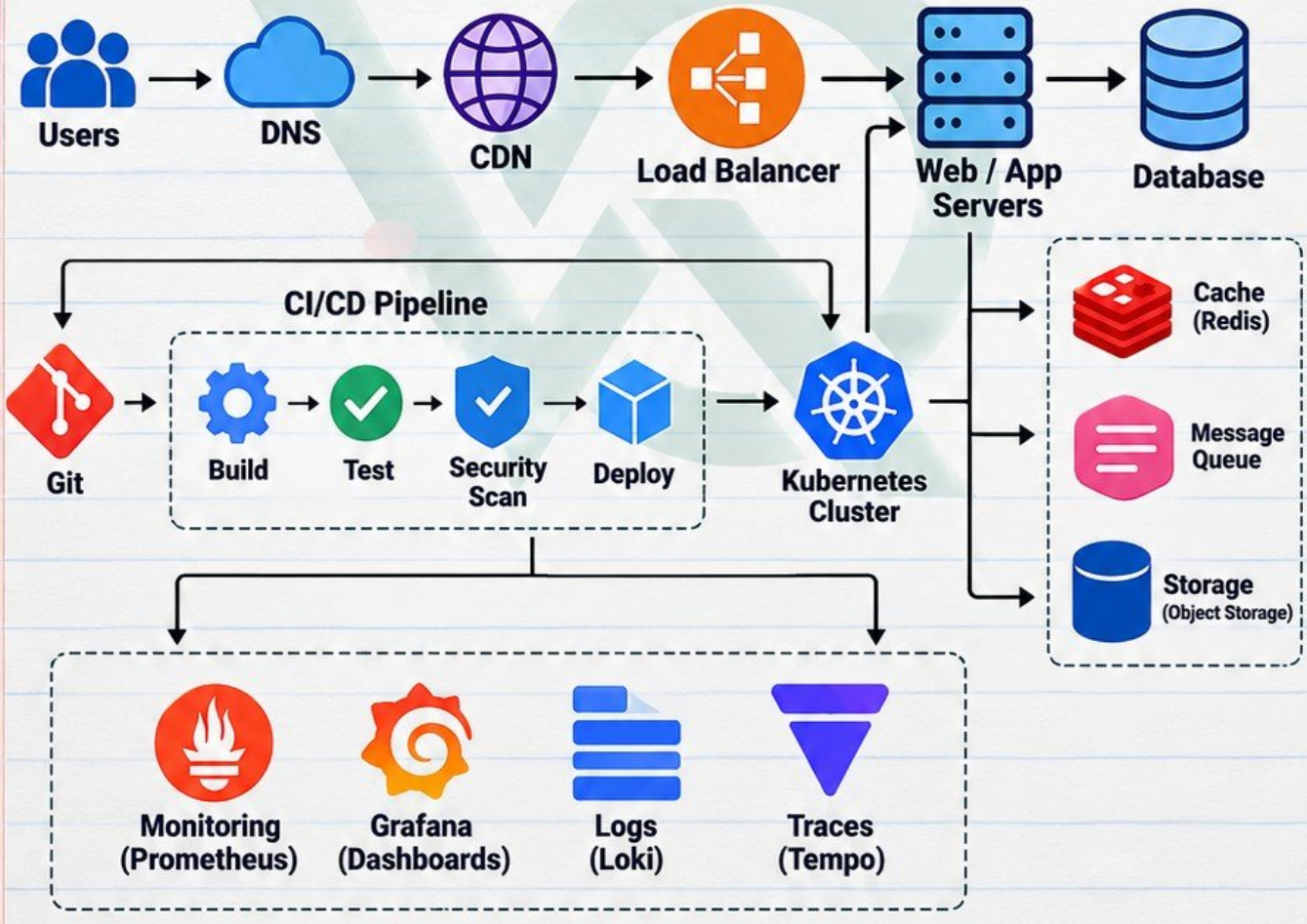
VERIQTA

DevOps Architecture Diagrams

30 Real Systems Explained Step by Step



@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

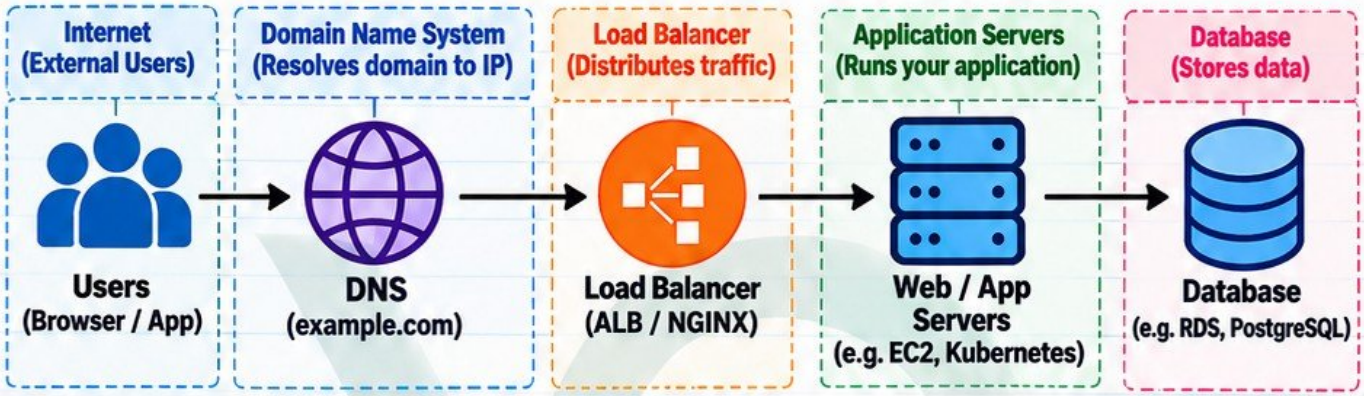
How to Read a Production Architecture Diagram

01

VERIQTA

A production architecture diagram shows how all the components of an application work together in a real environment. It helps you understand how requests flow, how the application is deployed, where data moves, and where failures can happen.

Example: Simple Web Application Architecture



Request Flow

- Shows how a user request moves through the system.
- In this example: User → DNS → Load Balancer → Application → Database.
- Helps you understand how the application serves users.

Deployment Flow

- Shows how code and configuration are delivered to the infrastructure.
- Usually: Developer → Git → CI/CD Pipeline → Build → Deploy → Servers/Cluster.
- Helps you understand how new versions reach the application.

Data Flow

- Shows how data is created, read, updated and stored.
- In this example: Application → Database and back to Application.
- Helps you ndentify stateful components and data dependencies.

Failure Boundaries

- Shows where failures can happen and how they impact the system.
- Examples: DNS failure, load balancer failure, server failure, database failure.
- Helps you plan redundancy, monitoring and recovery.

Key Things to Identify

1. Entry points (where users or external systems connect).
2. Trust boundaries (internet vs private network).
3. Dependencies (what each component relies on).
4. Stateful components (databases, storage, message queues).
5. Single points of failure (components without redundancy).

Junior Engineer Tips

- Always follow the arrows to understand the flow.
- Identify which components are inside your cloud network and which are public.
- Look for components with persistent data.
- Check what happens if a component is unavailable.
- Use the diagram to understand where to investigate when a problem occurs.

Practice This

- Look at the diagram above and list all the components.
- For each component, identify what would happen if it fails.

From a Single Server to a Three-Tier Application

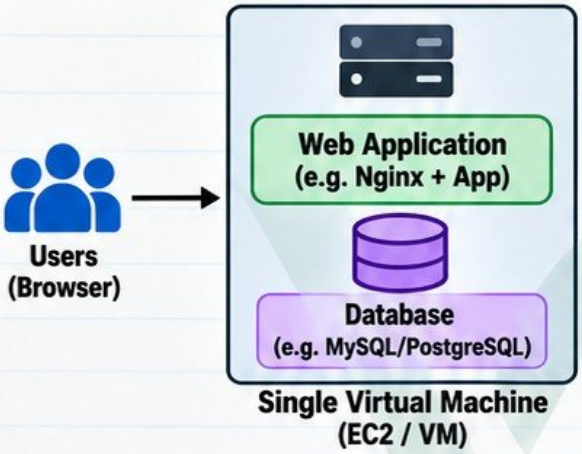
02

VERIQTA

Both architectures can run the same application, but they solve different problems. A single server is simple and suitable for small projects. A three-tier architecture separates components for better control, scalability, and reliability, but introduces network dependencies.

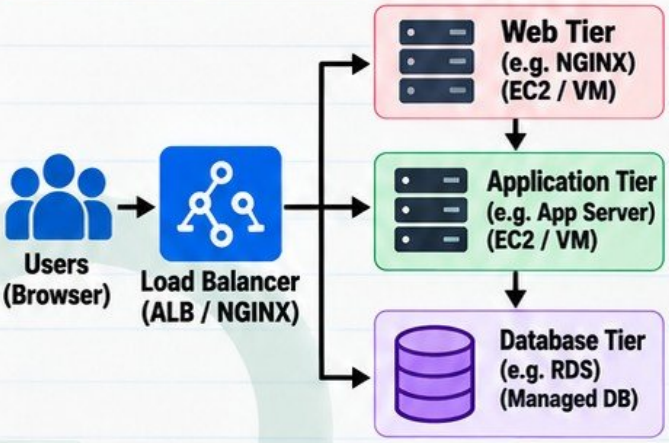
System 01: Single Server

A single virtual machine hosting the web application and a local database.



System 02: Three-Tier Application

Separate web, application, and database layers, usually on different servers or services.



Advantages of Single Server

- Simple to set up and manage.
- Lower cost for small projects.
- All components are on one machine.
- No network latency between app and DB.

Disadvantages of Single Server

- Single point of failure.
- Limited scalability.
- Application and database share resources.
- Harder to manage as the application grows.
- Changes can affect the entire server.

Advantages of Three-Tier

- Separates concerns (web, app, database).
- Independent scaling for each tier.
- Better security and network control.
- Easier deployment and maintenance.
- More reliable and production-ready.

Disadvantages of Three-Tier

- More complex to set up and manage.
- Higher cost (multiple servers or services).
- Network latency between tiers.
- More components to monitor and secure.
- Reguires proper network configuration (security groups, firewalls).

When to Use

Single Server	• Learning and development • Small projects • Proof of concept
Three-Tier	• Production applications • Growing user base • Need for scalability and control • Team-based development

Key Takeaways

1. Separation improves deployment control, scalability, and reliability.
2. Each tier can be updated independently.
3. Separation introduces network dependencies, which can become a source of failure.
4. Choose the right architecture based on your application's current needs and future growth.

Practice This

- Draw both architectures and list their components.
- Identify at least two potential failure points in each architecture.

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

How Load Balancers Keep Applications Available

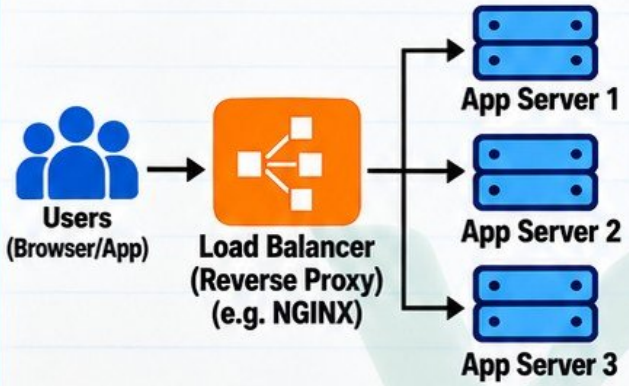
03

VERIQTA

A load balancer distributes incoming traffic to multiple application instances, improves availability, and helps the application handle failures and traffic spikes.

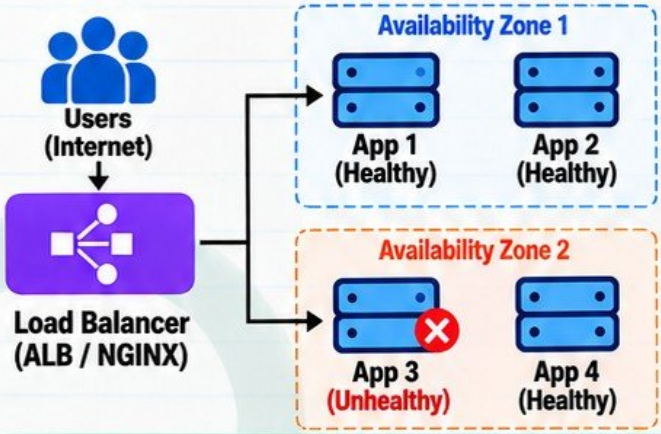
System 03: Reverse Proxy

A reverse proxy sits in front of your application servers and routes client requests to the appropriate backend servers.



System 04: Highly Available Application

A load balancer distributes traffic across multiple instances in different availability zones. It performs health checks and only sends traffic to healthy instances.



How a Load Balancer Works

1. Receives incoming user requests.
2. Uses a routing algorithm (e.g. round-robin, least connections).
3. Sends the request to a healthy backend server.
4. Performs health checks on each instance.
5. Stops sending traffic to unhealthy instances.
6. Distributes traffic across multiple instances.

What Happens When an Instance Becomes Unhealthy

1. Health check fails (e.g. HTTP, TCP, or custom check).
2. Load balancer marks the instance as unhealthy.
3. Stops sending new requests to that instance.
4. Existing connections are drained (connection draining) to allow in-flight requests to finish.
5. Traffic is redistributed to healthy instances.
6. If too many instances are unhealthy, application capacity is reduced, which can cause higher latency or errors.

Load Balancing Algorithms

Algorithm	How it works
Round Robin	Sends requests to each server in order.
Least Connections	Sends to server with the fewest active connections.
IP Hash	Routes same user (IP) to same server.
Weighted	Sends more traffic to higher capacity servers.

Common Problems and Troubleshooting

Issue	Possible Cause	What to Check
High latency	Too few healthy instances	Load balancer metrics, instance health checks
HTTP 502/503 errors	All or most instances unhealthy	Health check logs, application logs
Uneven traffic distribution	Incorrect algorithm or weights	Load balancer configuration
Users still routed to unhealthy server	Health check misconfigured	Health check path, interval, timeout
Connection drops	No or short draining period	Connection draining settings

Key Takeaways

1. Load balancers improve availability and handle traffic spikes.
2. Health checks and connection draining reduce downtime.
3. Multiple instances across availability zones protect against failures.
4. If many instances are unhealthy, application capacity is reduced.

Practice This

- Draw both architectures and label each component.
- Identify what happens when one instance becomes unhealthy.
- List at least two things you would check if users get 502 errors.

DNS, TLS, CDN, and the Journey of an HTTPS Request

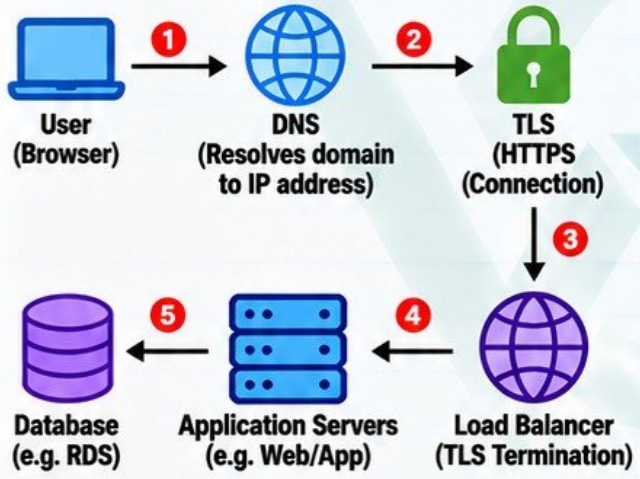
04

VERIQTA

When you visit a website using HTTPS, several components work together behind the scenes. This page shows the request journey and how DNS, TLS, CDN, and the origin server fit together.

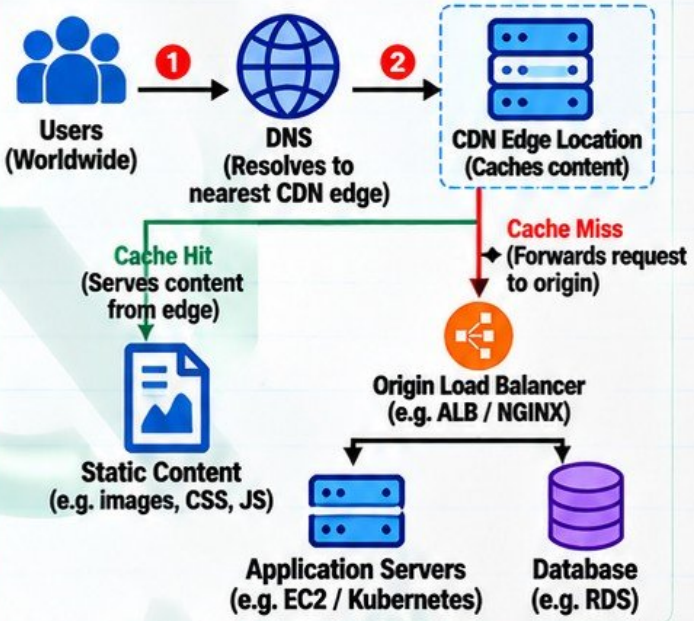
System 05: DNS, HTTPS and Origin Routing

The complete journey from your browser to the application, including DNS resolution, TLS handshake, and routing to the origin server.



System 06: CDN with Edge Caching

A content delivery network (CDN) serves content from edge locations worldwide and forwards requests to the origin server when needed.



Steps in an HTPS Request (System 05)

1. **DNS Resolution** – Your browser asks DNS for the IP address of the domain name.
2. **TLS Handshake** – Your browser and server establish an encrypted HTTPS connection.
3. **Load Balancer** – Receives the request and sends it to a healthy application server (TLS may terminate here).
4. **Application Server** – Processes the request.
5. **Database** – Retrieves or stores data for the application.

How a CDN Works (System 06)

1. DNS redirects you to the nearest CDN edge location.
2. The CDN edge checks if the content is cached.
3. If cached, it serves the content directly (faster).
4. If not cached, it forwards the request to the origin load balancer.
5. The origin load balancer sends the request to application servers, which may access the database.

Common Failures and How to Investigate

Component	Common Issue	How to Investigate
DNS	Domain not resolving or wrong IP	`nslookup domain.com` or `dig domain.com`
TLS	Expired or invalid certificate	`openssl s_client -connect domain.com:443`
CDN	Stale or incorrect cached content	Check CDN cache settings and invalidation rules
Load Balancer	Sending traffic to unhealthy servers	Check health checks, target status, and logs
Origin Server	Outage or high latency	Check server logs, metrics, and application status

Key Differences

	System 05 (DNS, HTTPS, Origin)	System 06 (CDN with Edge Caching)
Primary purpose	Deliver dynamic application traffic	Serve static and dynamic content globally
Components	DNS, TLS, Load Balancer, App Servers, Database	DNS, CDN Edge, Origin Load Balancer, App Servers
Performance	Good for single region or small scale	Faster for global users due to edge locations
Complexity	Simpler	More complex (more components)

Practice This

- Draw both architectures and label each component.
- Identify at least two failure points in each system.
- Explain what would happen if the CDN origin server is down.

The Architecture Behind a Real CI Pipeline

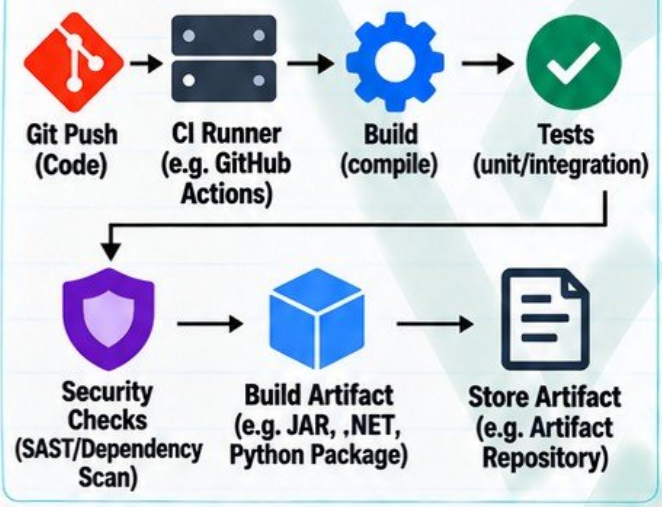
05

VERIQTA

A CI pipeline automatically builds, tests, scans, and packages your application every time you push code. A successful build does not mean the application is safe, secure, or ready for deployment.

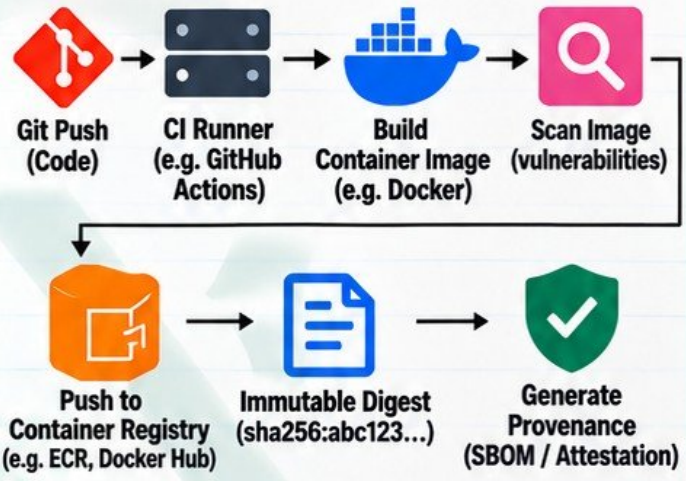
System 07: Code to Build Artifact

A basic CI pipeline that builds, tests, scans and produces an artifact (binary or package).



System 08: Container Image Pipeline

Builds a container image, scans it, stores it in a registry with an immutable digest, and generates provenance (build metadata).



Why Use a CI Pipeline?

- Automatically checks code quality.
- Catches errors early.
- Runs tests and security scans.
- Produces a consistent, versioned artifact.
- Reduces human error.
- Speeds up delivery.
- Provides traceability and audit history.

Common Misconceptions

- A successful build means the application is safe.
- Tests always catch all bugs.
- Security scans are not necessary.
- A container image is ready just because it built.
- You can use the latest tag in production.
- Scanning once is enough.
- The artifact does not need provenance or metadata.

Key Pipeline Stages (System 07)

Stage	What It Does	Examples
Build	Compiles the code	mvn package, npm build
Test	Runs automated tests	unit, integration, e2e tests
Security Scan	Checks for vulnerabilities in code and dependencies	SAST, dependency scan (e.g. Trivy, Snyk)
Artifact	Creates a versioned package	JAR, WAR, .NET, Python package

Key Pipeline Stages (System 08)

Stage	What It Does	Examples
Build Image	Creates a container image from Dockerfile	docker build
Scan Image	Checks for OS and application vulnerabilities	Trivy, Grype
Push to Registry	Stores image with a unique immutable digest	Amazon ECR, Docker Hub
Provenance	Records build metadata (SBOM, signatures)	SLSA, cosign, in-toto

Junior Engineer Tips

1. Always review pipeline logs when a build fails.
2. Use immutable image digests instead of latest tags.
3. Include both code and image security scanning.
4. Keep artifacts and images properly versioned.

What Can Go Wrong?

1. Build succeeds but tests are skipped.
2. Vulnerabilities found in dependencies or base image.
3. Image pushed with critical vulnerabilities.
3. Wrong or mutable tag deployed to production.
4. Missing provenance or traceability information.

Practice This

1. Set up a simple CI pipeline for a sample application.
2. Add a security scan stage.
3. Build and push a container image to a registry.
4. Verify the image digest and check the scan results.

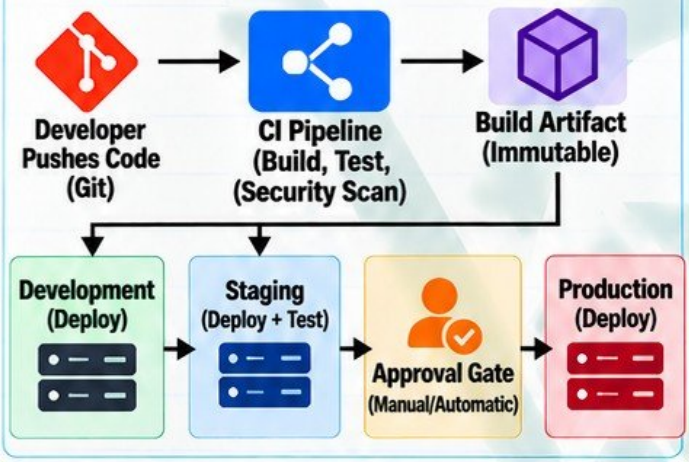
How Continuous Deployment Actually Reaches Production 06

VERIQTA

There are different ways to deploy to production. This page compares a traditional CI/CD pipeline (push-based deployment) with a GitOps model (pull-based reconciliation) and explains how each one works, their differences, and what can go wrong.

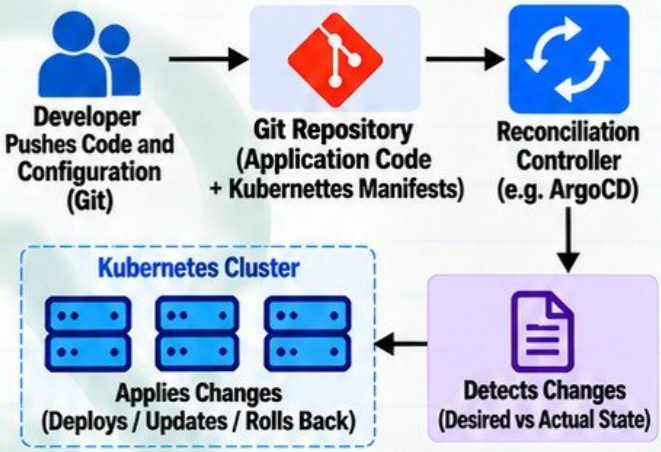
System 09: CI/CD Deployment (Push-Based)

Code is pushed through a pipeline that builds, tests and deploys to development, staging, and production with approval gates.



System 10: GitOps Deployment (Pull-Based)

Code and configuration are stored in Git. A reconciliation controller continuously checks the desired state and applies changes to the Kubernetes cluster.



CI/CD Flow Steps (System 09)

1. Developer pushes code to Git.
2. CI pipeline builds the code and runs tests.
3. Security scans check for vulnerabilities.
4. Build artifact is created (e.g. container image).
5. Deploys to development environment.
6. Deploys to staging environment and runs tests.
7. Approval gate (manual or automatic).
8. Deploys to production environment.

GitOps Flow Steps (System 10)

1. Developer pushes code and manifests to Git.
2. Reconciliation controller detects changes.
3. Compares desired state in Git with actual state in the cluster.
4. Applies the changes automatically.
5. Continuously monitors for drift.
6. Rolls back if the desired state is changed.
7. Keeps the cluster aligned with what is in Git.

CI/CD vs GitOps Comparison

Aspect	CI/CD (Push-Based)	GitOps (Pull-Based)
How changes reach production	Pipeline pushes changes to environments.	Controller pulls changes from Git and applies them.
Source of truth	Pipeline and artifact repository.	Git repository.
Approval process	Built into the pipeline (manual or automatic).	Handled through Git changes (pull requests).
Drift detection	Not automatic.	Continuous reconciliation.
Rollback	Pipeline triggers rollback.	Revert Git change, controller applies it.
Best for	Simple and fast deployments.	Consistent, auditable, large-scale environments.

Common Issues and How to Investigate

Issue	Possible Cause	What to Check
Deployment fails	Tests fail, misconfiguration, missing secrets.	Pipeline logs, test results, configuration files.
Staging works but production fails	Environment differences, missing approvals.	Environment config, pipeline logs, approval settings.
Changes not applied in GitOps	Controller not running, invalid manifests.	Controller logs, Git repo, Kubernetes events.
Cluster drift	Manual changes in cluster.	Compare desired vs actual state, controller logs.
Rollback not working	Incorrect image tag or old configuration.	Pipeline history, Git history, cluster events.

Key Takeaways

1. CI/CD uses a push model. GitOps uses a pull model.
2. Both can deploy the same application successfully.
3. GitOps provides continuous drift detection.
4. Successful build does not mean the application is safe or ready for production.

Practice This

1. Draw both architectures and label each component.
2. Identify at least two failure points in each system.
3. Explain what happens when a deployment fails.
4. Decide which model (CI/CD or GitOps) is better for a large team and explain why.

Blue-Green and Canary Deployment Architecture

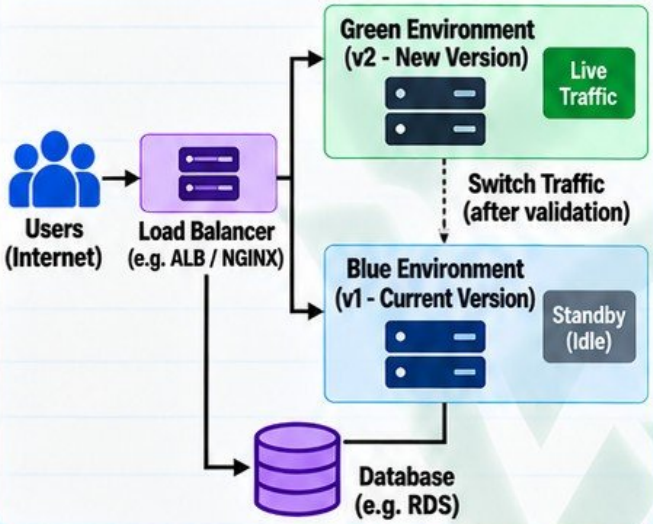
07

VERIQTA

Both blue-green and canary deployments reduce risk when releasing new versions. Blue-green switches all traffic at once, while canary gradually increases traffic to the new version based on real metrics. Application rollback does not automatically reverse database schema changes.

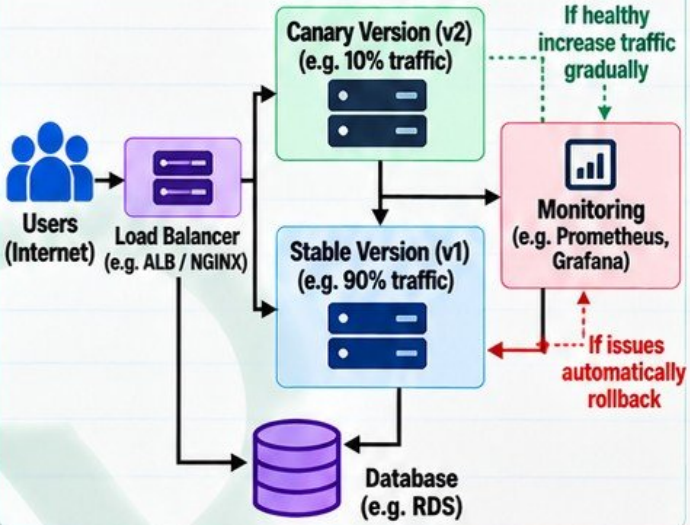
System 11: Blue-Green Deployment

Two identical environments (blue and green) are used. Only one environment receives live traffic. Traffic is switched to the new version when ready.



System 12: Canary Deployment

A small percentage of traffic is sent to the new version (canary). Metrics are monitored. If the new version is healthy, traffic is gradually increased.



Advantages of Blue-Green Deployment

- Simple and fast traffic switch.
- Reduced risk of downtime.
- Easy and quick rollback (switch back).
- Both environments are identical.
- Useful for high-risk releases.

Disadvantages of Blue-Green Deployment

- Requires double infrastructure (higher cost).
- Database changes are shared (rollback is complex).
- Not ideal for gradual testing with real users.
- Idle environment still incurs cost.
- Requires automation for smooth switching.

Advantages of Canary Deployment

- Gradual traffic increase reduces risk.
- Real user metrics validate the new version.
- Automatic rollback can be configured.
- Less infrastructure cost than blue-green.
- Works well for large and complex applications.

Disadvantages of Canary Deployment

- More complex to set up and manage.
- Requires good monitoring and alerting.
- Traffic is split, so issues may still affect some users.
- Slower full rollout compared to blue-green.
- Requires well-defined success and rollback criteria.

Traffic Flow Comparison

Aspect	Blue-Green (System 11)	Canary (System 12)
Traffic Routing	100% to one environment	Small % to new version, then gradually increase
Rollback	Immediate switch back	Automatic or manual (gradual rollback)
Risk Level	Lower (tested before switch)	Lower (real user testing)
Infrastructure Cost	Higher (two environments)	Lower (shares resources)
Best Use Case	High-risk releases, fast switch required	Continuous delivery, large user base

Why Application Rollback Does Not Reverse Database Schema Changes

1. Database schema changes are persistent.
2. Application rollback only changes the application code.
3. Database migrations are usually applied separately (e.g. by a tool).
4. Rolling back the application does not automatically undo schema changes or data modifications.
5. You may need a separate rollback plan for database changes (e.g. backward-compatible migrations, manual rollback scripts).

Common Issues and How to Investigate

Issue	Possible Cause	What to Check
Users still hitting old version	DNS / load balancer config	Load balancer rules, DNS TTL
Errors after deployment	Database schema mismatch	Application logs, DB migration status
High error rate on canary	Bug in new version	Metrics, logs, APM traces
Rollback did not fix issue	Database or external dependency	DB changes, external services

Practice This

1. Draw both architectures and label each component.
2. Show the traffic flow for a successful deployment.
3. Identify at least two reasons why a rollback might still fail.
4. Explain how you would monitor a canary release.
5. List the checks you would perform before increasing canary traffic.

Docker Architecture: From Image to Running Application

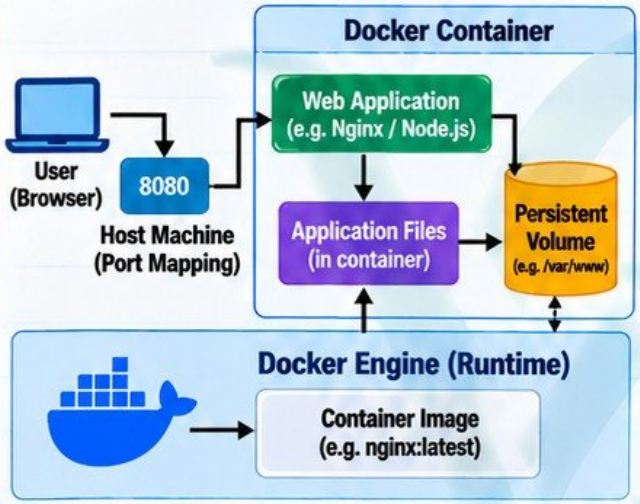
08

VERIQTA

Docker packages your application and its dependencies into containers, providing a consistent and isolated environment. This page shows a single-container application and a multi-container application, and explains how they work, communicate, and persist data.

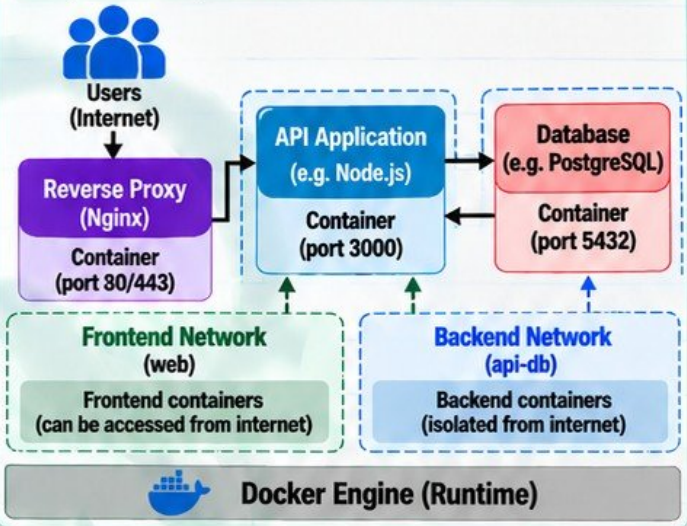
System 13: Single Container Application

A containerized web application with an image, runtime, network, and persistent volume.



System 14: Multi-Container Application

A multi-container application with a reverse proxy, API, database, and isolated networks.



How a Single Container Works (System 13)

1. You pull a container image from a registry (e.g. Docker Hub).
2. Docker creates a container from the image.
3. The container runs your application with all dependencies.
4. The container is connected to a network.
5. A volume is mounted for persistent data.
6. The host port is mapped so users can access the application.

How a Multi-Container App Works (System 14)

1. Users access the application through a reverse proxy.
2. The reverse proxy forwards requests to the API container.
3. The API container communicates with the database container using an internal network.
4. Containers are isolated using separate networks.
5. Data is stored in a persistent volume for the database.
6. Only the reverse proxy is exposed to the internet.

Key Docker Components

Component	Purpose
Image	Read-only template with application and dependencies.
Container	Running instance of an image.
Docker Engine	Builds, runs, and manages containers.
Network	Allows containers to communicate.
Volume	Stores persistent data outside the container.
Port Mapping	Maps container ports to host ports.

Key Differences (System 13 vs System 14)

Aspect	System 13 (Single Container)	System 14 (Multi-Container)
Architecture	One container	Multiple containers
Components	Web app + local storage	Reverse proxy, API, database
Networking	Single network	Separated frontend and backend networks
Data	Single volume	Separate volumes (e.g. database volume)
Exposure	App exposed (mapped port)	Only reverse proxy exposed
Use case	Simple applications	Real-world applications

Common Issues and How to Investigate

Issue	Possible Cause	What to Check
Container won't start	Image error or bad config	docker logs <container>
Cannot access app	Port not mapped or firewall	docker ps, docker port
Containers can't communicate	Wrong network or DNS	docker network inspect, ping, nslookup
Data is lost after restart	No volume or wrong mount	docker volume ls, docker inspect
Database connection fails	Wrong host/port or network	docker logs, container env
Container keeps restarting	App crash or health check fail	docker logs, docker inspect

Key Takeaways

1. An image is a template. A container is a running instance.
2. Use volumes for persistent data.
3. Use networks to control container communication.
4. Keep sensitive services (e.g. database) isolated from the internet.
5. Container restart does not delete data stored in a volume.
6. Always check logs and container configuration when troubleshooting.

Practice This

1. Draw both architectures and label each component.
2. Explain how the containers communicate in System 14.
3. Identify what happens if the database container restarts.
6. Show the command you would use to view container logs.

Kubernetes Architecture: From API Request to Running Pod

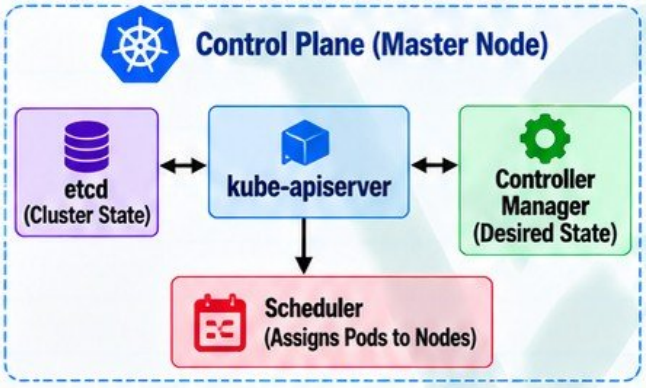
09

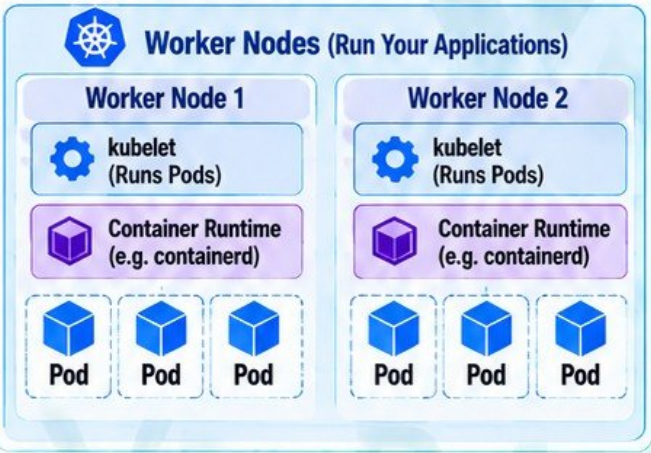
VERIQTA

This page shows how a Kubernetes cluster works and how an application request reaches a running Pod. It covers the control plane components, the worker nodes, and the resources used to deploy and expose an application. It also explains why a running Pod may still be unable to serve requests.

System 15: Kubernetes Cluster Architecture

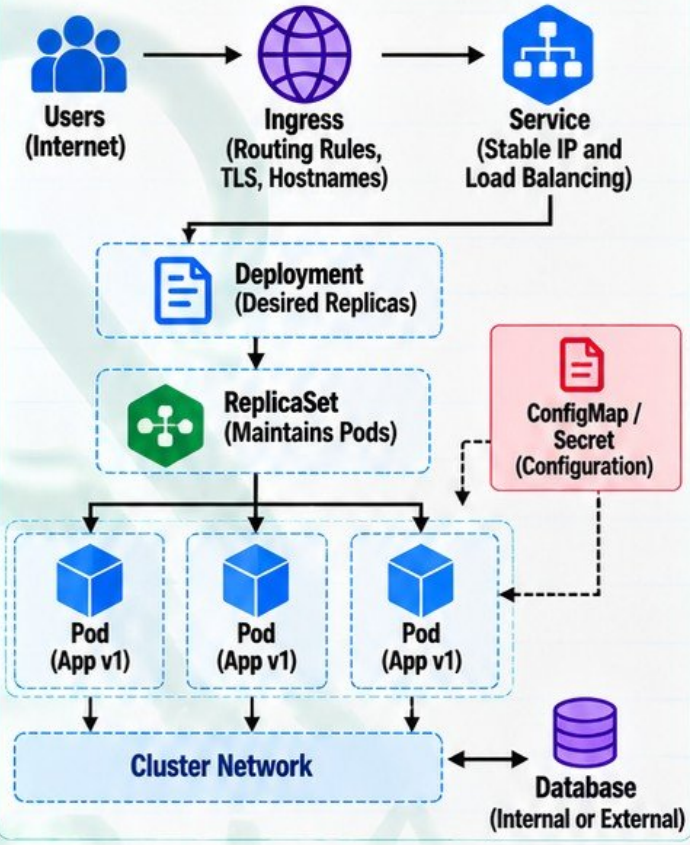
The control plane manages the cluster, while worker nodes run your applications. Components work together to keep the desired state.





System 16: Application Deployment and Request Flow

An application is deployed using a Deployment, which creates a ReplicaSet and Pods. A Service provides a stable endpoint and an Ingress exposes it to external users.



Key Control Plane Components (System 15)

Component	Purpose
kube-apiserver	Receives API requests and validates them.
etcd	Stores the cluster state (all objects).
controller manager	Runs controllers to keep the desired state.
scheduler	Assigns Pods to suitable worker nodes.
kubelet	Runs and monitors Pods on each node.
container runtime	Pulls images and runs containers.

Key Application Components (System 16)

Component	Purpose
Deployment	Defines the desired state for your application.
ReplicaSet	Ensures the correct number of Pods are running.
Pod	Runs one or more containers.
Service	Provides a stable endpoint and load balancing.
Ingress	Routes external traffic to the Service.
ConfigMap / Secret	Stores configuration and sensitive data.

How Reconciliation Works

1. You define the desired state (e.g. 3 replicas).
2. Controllers continuously compare actual vs desired state.
3. If a Pod fails, a new one is created automatically.
4. The cluster keeps trying until the desired state is reached.

Why a Running Pod May Still Not Serve Requests

1. Readiness probe is failing.
2. Service or Ingress is misconfigured.
3. Network policies are blocking traffic.
4. The application inside the Pod is unhealthy.
5. Dependent services (e.g. database) are unavailable.

Practice This

1. Draw both architectures and label each component.
2. Show the flow of an API request from the internet to a running Pod.
3. Identify at least two reasons why a running Pod may not serve requests.
4. Explain what happens when a node fails.
5. Show how the controller replaces a failed Pod.

Kubernetes Autoscaling and Workload Placement

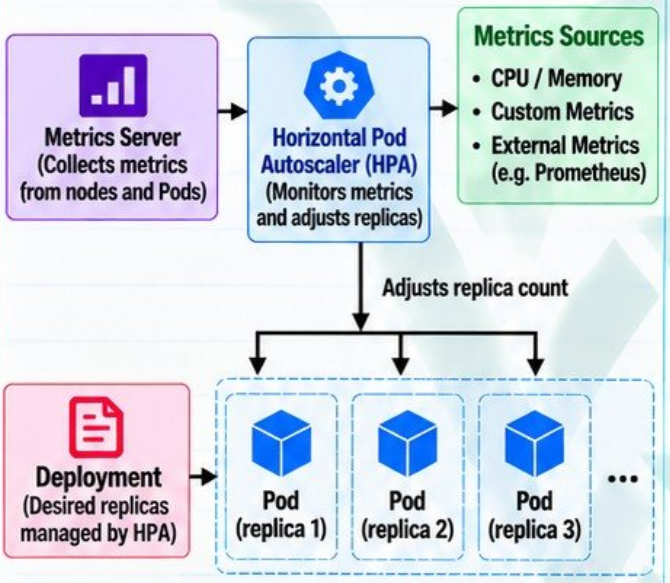
10

VERIQTA

Autoscaling helps your application handle changing demand, while workload placement decides where Pods run in the cluster. This page explains how Pod autoscaling, node autoscaling, scheduling, and availability-zone placement work together, and their limitations.

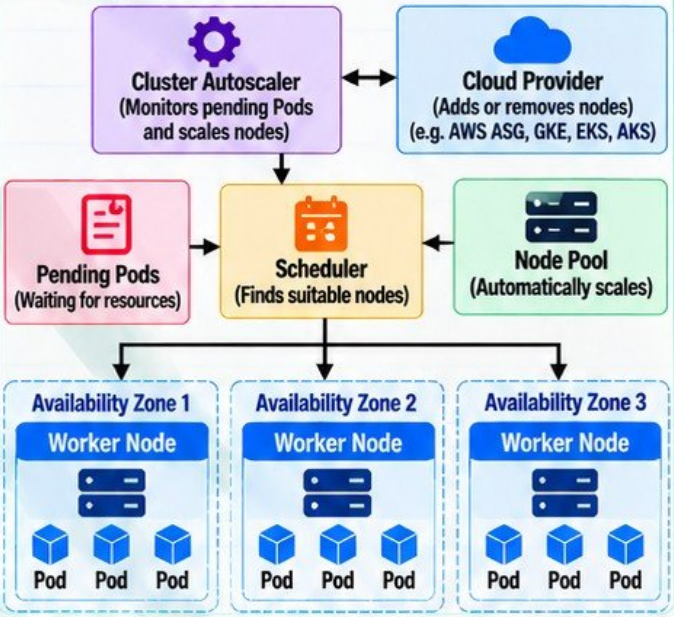
System 17: Horizontal Pod Autoscaler (HPA)

HPA automatically adjusts the number of Pod replicas based on metrics such as CPU, memory, or custom application metrics.



System 18: Cluster Node Autoscaling and Workload Placement

The cluster automatically adds or removes nodes based on resource demand. The scheduler places Pods on suitable nodes, considering resource requests, constraints and availability zones.



Key Components (System 17 - HPA)

Component	Purpose
Metrics Server	Collects CPU and memory metrics from nodes and Pods.
HPA	Monitors metrics and adjusts the number of Pod replicas.
Deployment	Defines the desired state (replicas managed by HPA).
Metrics Sources	CPU, memory, custom metrics or external metrics (e.g. Prometheus).
Pods	Application replicas that scale in or out automatically.

Key Components (System 18 - Node Autoscaling)

Component	Purpose
Cluster Autoscaler	Adds or removes nodes based on pending Pods and resource demand.
Cloud Provider	Creates or terminates nodes (e.g. AWS, GCP, Azure).
Scheduler	Places Pods on suitable nodes based on resources, labels, taints, and constraints.
Node Pool	Group of nodes that scales automatically.
Worker Node	Runs application Pods.
Availability Zone	Nodes are distributed across zones for high availability.

Benefits

1. Handles traffic spikes automatically.
2. Reduces manual scaling.
3. Improves resource utilization.
4. Maintains application availability.
5. Supports multiple metric types (CPU, memory, custom).
6. Spreads workloads across availability zones.

Limitations and Common Misconceptions

1. Adding Pods cannot solve insufficient node capacity.
2. Autoscaling does not fix a slow database.
3. HPA only scales based on metrics, it does not know if your application is healthy.
4. Node autoscaling takes time (new nodes are not instant).
5. Poor resource requests and limits can cause scheduling failures.
6. Autoscaling cannot fix code, configuration, network or external dependency issues.

Practice This

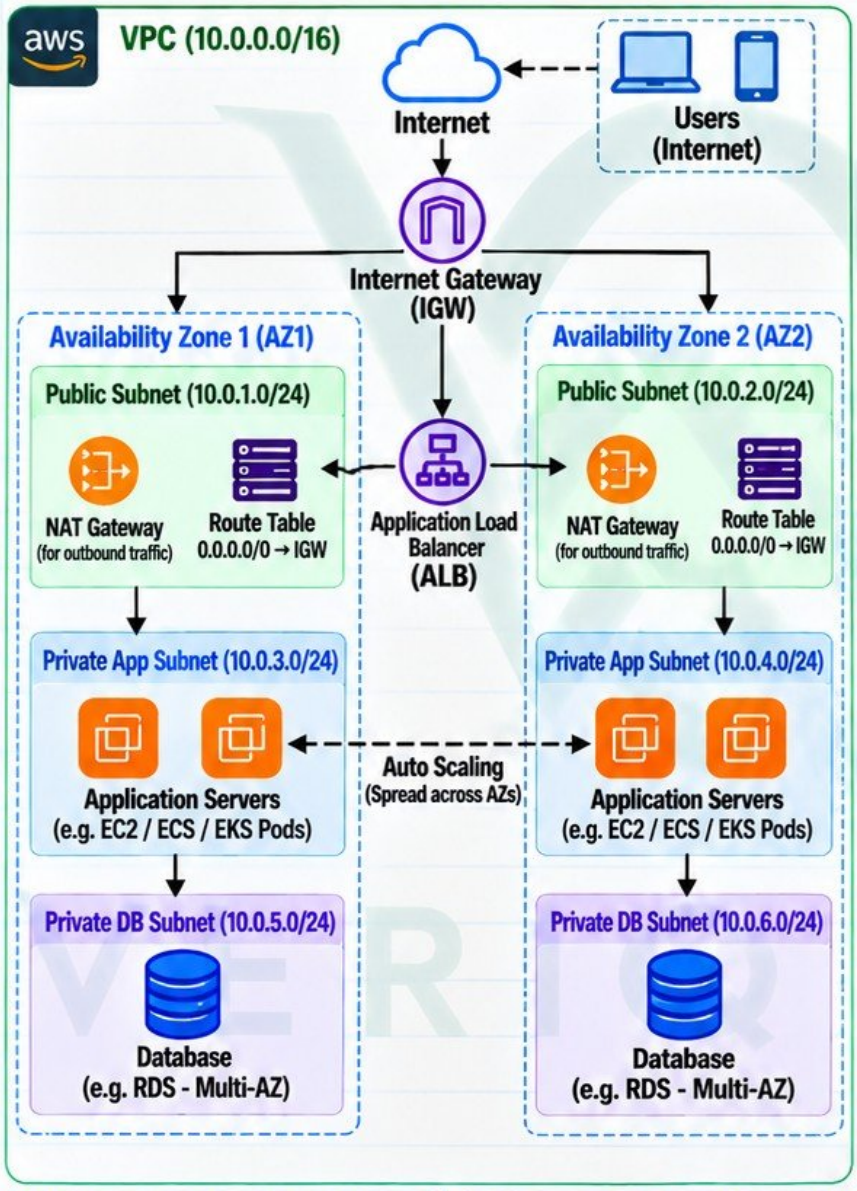
1. Draw both architectures and label each component.
2. Show how HPA increases and decreases the number of replicas.
3. Show how the cluster autoscaler adds a new node when Pods are pending.
4. Identify at least three reasons why adding Pods does not solve a database problem.
5. Explain what happens when there is not enough node capacity.
6. List the metrics you would use for HPA in a real application.

A Secure AWS VPC Architecture

VERIQTA

This page shows a secure and highly available VPC design with public and private subnets across two availability zones. It explains how traffic flows, the role of each component, and important security considerations.

System 19: AWS VPC Architecture (Two Availability Zones)



Key Components

Component	Purpose
VPC	Isolated network for your resources.
Internet Gateway (IGW)	Allows inbound and outbound internet traffic for public subnets.
Public Subnet	Resources with direct internet access (e.g. ALB, NAT).
NAT Gateway	Provides outbound internet access for private subnets.
Application Load Balancer (ALB)	Distributes incoming traffic to application servers across AZs.
Private App Subnet	Holds application servers (EC2/ECS/EKS) with no direct internet access.
Private DB Subnet	Holds database resources (e.g. RDS) with no direct internet access.
Route Tables	Define where traffic goes (e.g. 0.0.0.0/0 → IGW or Nat).
Security Groups	Stateful firewall rules for instances and load balancers.
Availability Zone (AZ)	Separate data centres for high availability.

Traffic Flow (Inbound and Outbound)

1. Users send requests over the internet.
2. Traffic enters the VPC through the Internet Gateway (IGW).
3. The Application Load Balancer (ALB) in public subnets distributes traffic to application servers in private subnets.
4. Application servers access the database in private DB subnets.
5. For outbound internet access (e.g. updates), private subnets use a NAT Gateway.
6. Responses go back through the ALB to users on the internet.

Route Tables (Example)

Subnet	Destination	Target
Public Subnet (AZ1 & AZ2)	10.0.0.0/16	local
	0.0.0.0/0	IGW
Private App Subnet (AZ1)	10.0.0.0/16	local
	0.0.0.0/0	NAT Gateway (AZ1)
Private App Subnet (AZ2)	10.0.0.0/16	local
	0.0.0.0/0	NAT Gateway (AZ2)
Private DB Subnet (AZ1 & AZ2)	10.0.0.0/16	local
	0.0.0.0/0	(no route or NAT)

Security Groups (Example)

Resource	Key Rules
ALB SG	Allow HTTP/HTTPS (80/443) from 0.0.0.0/0. Allow to App SG (target port).
App Server SG	Allow from ALB SG (e.g. 80). Allow outbound to DB SG (e.g. 3306). Allow outbound to internet (via NAT).
Database SG	Allow from App SG (e.g. 3306). No inbound from internet.

Important Points

1. Private subnets are not automatically secure. You must configure security groups, NACLs and route tables correctly.
2. The ALB is placed in public subnets so it can receive internet traffic.
3. Application and database resources are in private subnets with no direct internet access.
4. Use a NAT Gateway for outbound internet access (from private subnets).
5. Always deploy across multiple availability zones for high availability.
6. Troubleshoot connectivity by checking route tables, security groups, NACLs, and instance network settings.

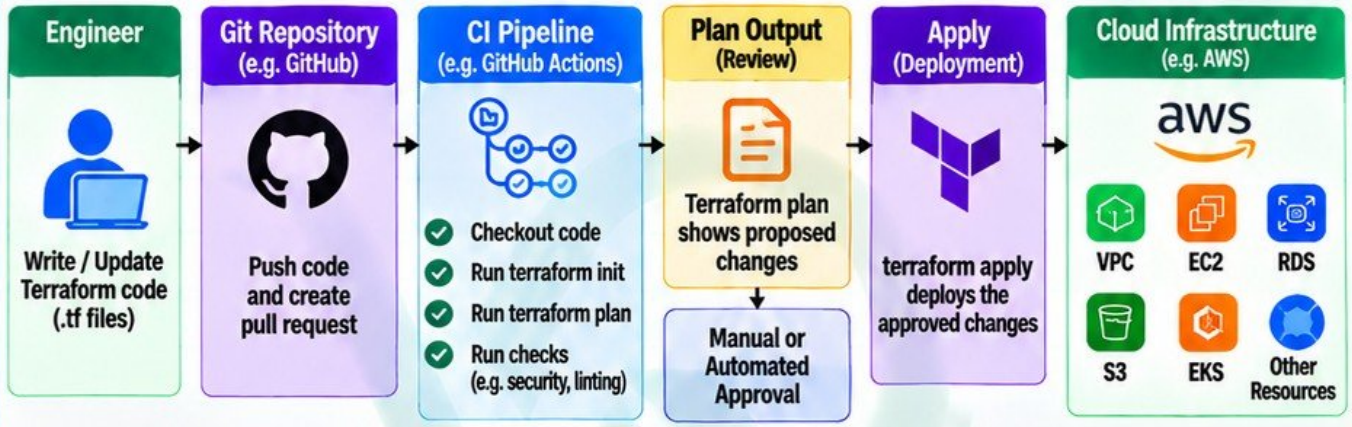
Infrastructure as Code and Remote State Architecture

12

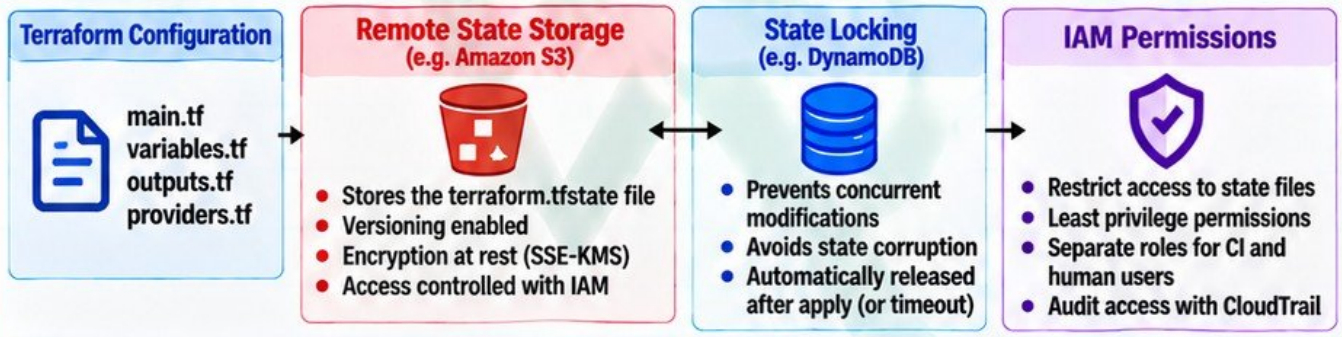
VERIQTA

This page shows a typical Infrastructure as Code (IaC) workflow using Terraform, including remote state storage, state locking, approvals, and the deployment of cloud infrastructure. It explains how the components work together and key operational considerations.

System 20: Terraform Infrastructure as Code Workflow



Remote State Architecture



State Recovery Considerations

- Use versioning to restore previous state versions.
- Keep state files encrypted and access controlled.
- Have a recovery plan for accidental state deletion or corruption.
- Understand how to manually unlock state if needed.

Common Issues and How to Investigate

Issue	Possible Cause	What to Check
Drift between code and real infrastructure	Manual changes in the cloud	Run terraform plan to see unexpected changes
Apply fails	Invalid configuration, missing permissions, or resource limits	Check CI logs, error messages, and IAM permissions
State file exposed	Public S3 bucket or weak IAM policies	Verify bucket policy, IAM roles and access logs
State lock stuck	Interrupted run or failed pipeline	Check DynamoDB lock table and manually remove stale lock
Concurrency conflicts	Multiple applies running at the same time	Use state locking and single pipeline runs
Unexpected resource changes	Code not reviewed or untracked changes	Review pull requests and plan output before apply

Key Takeaways

1. Always use a remote state backend (e.g. S3) with locking (e.g. DynamoDB).
2. Keep state files encrypted and access controlled.
3. Use a CI pipeline to run plan and require approval before apply.
4. Regularly review plan output to avoid unintended changes.
5. Understand and monitor drift between code and real resources.
6. Have a state recovery plan and know how to unlock a stuck state.
7. Restrict IAM permissions and use least privilege.
8. Infrastructure changes should always be reviewed, especially in production.

Practice This

1. Draw the full workflow and label each component.
2. Show where the state file is stored and how locking works.
3. Identify what happens if two people run apply at the same time.
4. Explain how you would recover from a deleted or corrupted state file.
5. Show how you would configure IAM permissions for the state backend.
6. List the checks you would perform before approving an apply.
7. Identify at least three reasons why drift can occur.
8. Explain how you would troubleshoot a failed apply in the CI pipeline.

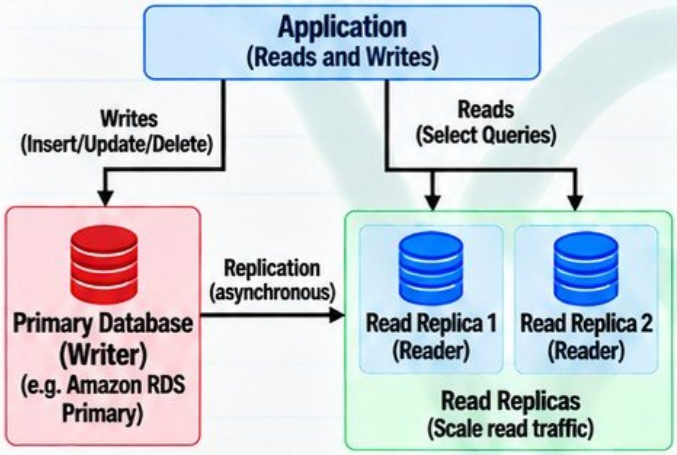
Database Availability, Replication, and Recovery

VERIQTA

This page covers database replication for read scalability and high availability, and backup strategies for recovery. It shows a primary database with read replicas, and a multi-availability-zone database with standby, automated backups, and point-in-time recovery.

System 21: Primary Database with Read Replicas

A primary database handles all write operations, while read replicas handle read traffic. Replication keeps the replicas in sync, but there can be a small delay (replication lag).

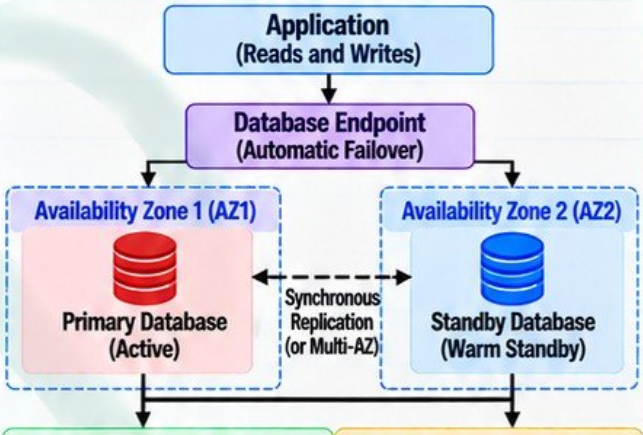


Replication Lag

There can be a delay of a few seconds to minutes before changes appear on replicas.

System 22: Multi-AZ Database with Failover and Recovery

A database is deployed across multiple availability zones with a standby for high availability. Automated backups are stored for recovery and point-in-time restore.



Automated Backups
(Stored in separate storage)

- Daily automated backups
- Transaction logs
- Stored in durable storage (e.g. Amazon S3)

Point-in-Time Recovery
(Restore to a specific time)

- Restore to any point in time
- Uses backups and logs
- Recover from accidental deletion or corruption

Key Components (System 21)

Component	Purpose
Primary Database	Handles all write operations.
Read Replica	Handles read queries to reduce load on primary.
Replication	Copies changes from primary to replicas.
Replication Lag	Delay between the primary and replicas.
Read Routing	Application or proxy routes read queries to replicas.

Key Components (System 22)

Component	Purpose
Primary Database (AZ1)	Active database handling read and write requests.
Standby Database (AZ2)	Synchronous replica for high availability (failover).
Multi-AZ	Deploys database across multiple availability zones.
Automated Backups	Regular backups stored in durable storage.
Point-in-Time Recovery	Restore database to a specific time using backups and logs.

Replication vs Backups vs High Availability vs Disaster Recovery

Concept	What It Means	Key Points
Replication	Copies data from primary to replicas (usually for reads).	• Near real-time • Can have replication lag • Not a backup
Backups	Point-in-time copies of data for recovery.	• Stored separately • Used to restore after data loss • Not for read scaling
High Availability (HA)	Keeps the database running during failures.	• Uses replicas/standby • Automatic or manual failover • Minimizes downtime
Disaster Recovery (DR)	Recovers from major disasters (e.g. region failure, data loss).	• Uses backups and/or cross-region replication • Involves restoring infrastructure and data • Longer recovery time

Common Issues and How to Investigate

Issue	Possible Cause	What to Check
Replication lag	High write load, large transactions	Check replica lag metrics (e.g. CloudWatch, performance insights)
Replica not receiving data	Network issue, replication error	Check replication status and logs
Failover not working	Misconfiguration	Check Multi-AZ settings and failover status
Backup failed	Permission or storage issue	Check backup logs and IAM permissions
Cannot restore to point in time	Logs not available or retention expired	Check backup retention period and log availability

Key Takeaways

1. Replication improves read scalability but can have lag.
2. Backups are for recovery, not for read scaling.
3. High availability keeps your database running during failures.
4. Disaster recovery helps you recover from major disasters.

Practice This

1. Draw both architectures and label each component.
2. Explain the flow of read and write requests.
3. Identify potential failure points and how to detect them.
4. Explain the difference between replication, backups, HA and DR.
5. Show how you would investigate and fix a replication lag issue.
6. Show the steps to restore a database to a specific point in time.

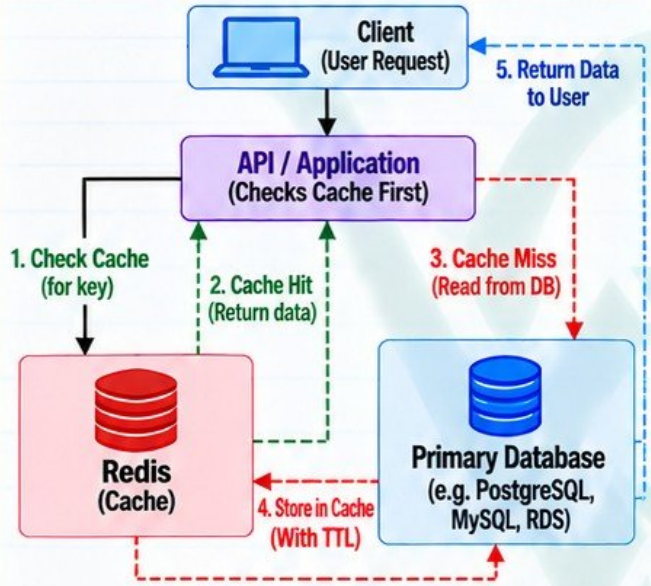
Redis Caching and Background Job Architecture

VERIQTA

This page shows how Redis is used for caching to improve application performance and how background jobs are processed asynchronously. It covers the main components, data flow, and common issues you will encounter in production.

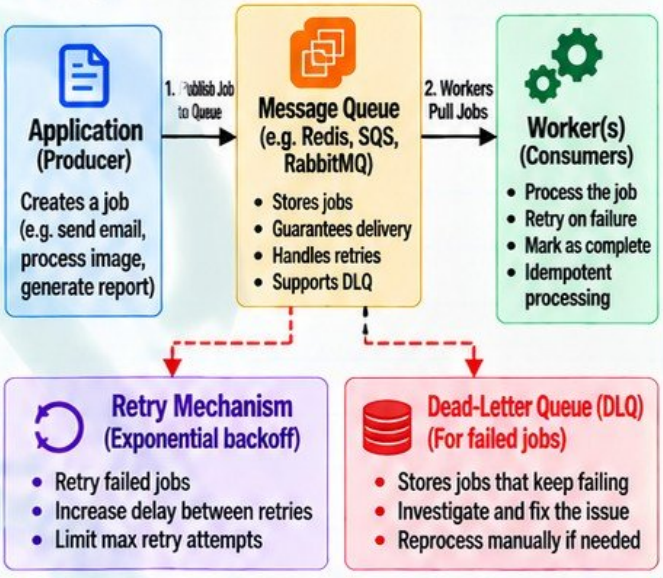
System 23: Cache-aside Architecture with Redis

The application checks Redis for data. If it is not in the cache (cache miss), it reads from the primary database, returns the data to the user and stores a copy in Redis for future requests.



System 24: Asynchronous Job Processing Architecture

Background jobs are placed in a queue and processed by workers independently of the main application. This allows time-consuming tasks to run asynchronously with retries and a dead-letter queue for failed jobs.



Key Points (System 23)

1. Cache-aside pattern gives better performance and reduces database load.
2. Data is stored in Redis with a time-to-live (TTL).
3. Cache miss triggers a read from the primary database.
4. After reading from the database, the data is stored in Redis for future requests.
5. The cache is updated or invalidated when data changes.

Key Points (System 24)

1. Jobs are created by the application and sent to a queue.
2. Workers pull and process jobs asynchronously.
3. Failed jobs are retried with exponential backoff.
4. Jobs that keep failing are moved to a dead-letter queue (DLQ).
5. Workers should be idempotent to avoid duplicate execution.

Common Issues in Caching (System 23)

Issue	Explanation
Cache stampede	Many requests hit the database at the same time when the cache expires.
Stale data	Cache contains old data after a database update.
Cache penetration	Requests for non-existent data cause repeated database queries.
Cache eviction	Important keys are removed due to memory limits.
Inconsistent data	Cache and database are not updated together.

Common Issues in Job Processing (System 24)

Issue	Explanation
Duplicate execution	The same job is processed more than once.
Worker backlog	Jobs pile up and take too long to process.
Failed jobs	Jobs fail repeatedly and move to the DLQ.
Lost jobs	Jobs are not delivered due to misconfiguration or outages.
Poison message	A bad job keeps failing and blocks the queue.

Key Takeaways

1. Redis caching improves performance but introduces challenges iike stale data and cache stampedes.
2. Background jobs help handle long-running tasks without blocking the main application.
3. Use retries and a dead-letter queue to handle failed jobs.
4. Monitor cache hit rate, queue length, worker health, and failed jobs.
5. Design for idempotency to avoid duplicate job execution.

Best Practices

1. Set appropriate TTL for cached data.
2. Use cache invalidation after updates.
3. Use a retry strategy with exponential backoff.
4. Monitor cache and queue metrics.
5. Design workers to be idempotent.

Practice This

1. Draw both architectures and label each component.
2. Show the flow of a cache hit and a cache miss.
3. Show the flow of a job from creation to successful processing.
4. Identify what happens during a cache stampede.
5. Explain how you would handle a failed job and reprocess it.

Microservices and Event-Driven Architecture

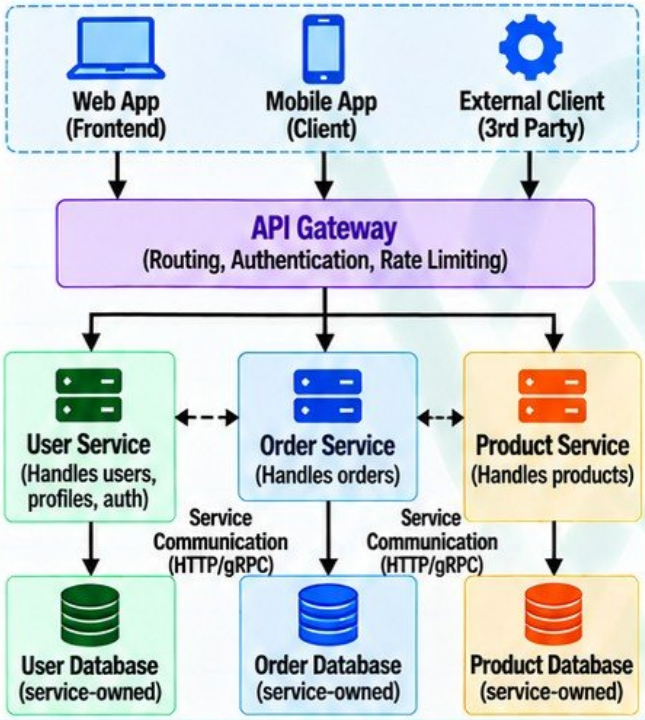
15

VERIQTA

This page shows two common modern architectures: microservices with an API gateway and service-owned data, and an event-driven system for asynchronous processing. It covers how the components work together and key operational considerations.

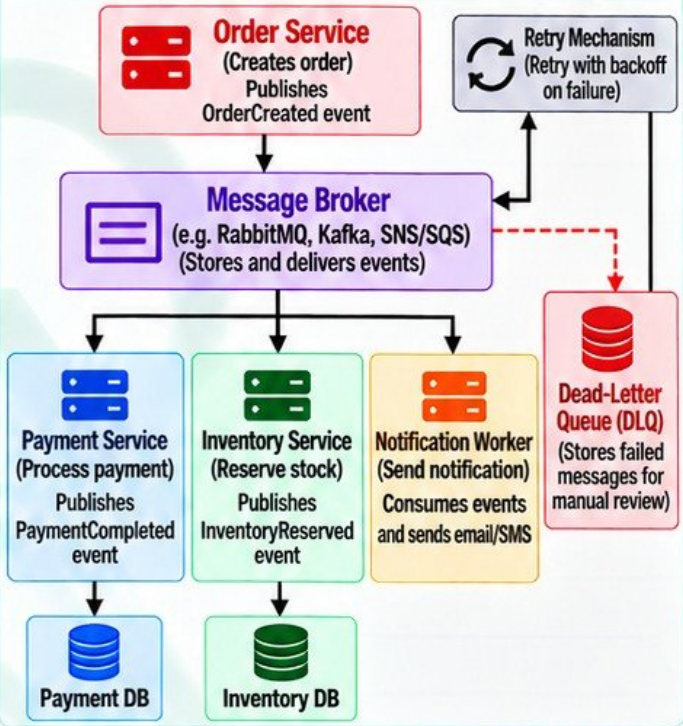
System 25: Microservices Architecture

An API gateway receives client requests and routes them to independently deployed services. Each service owns its own data.



System 26: Event-Driven Order Processing

An order is created and processed asynchronously using an event broker. Services react to events independently with retries and a dead-letter queue for failed messages.



Key Components (System 25)

Component	Purpose
Client	Sends requests to the API gateway.
API Gateway	Routes requests, handles authentication, rate limiting and TLS termination.
User Service	Manages user data and business logic.
Order Service	Manages order data and business logic.
Product Service	Manages product data and business logic.
Service Communication	HTTP or gRPC between services.
Databases	Each service owns its own database.

Key Components (System 26)

Component	Purpose
Order Service	Creates an order and publishes an event.
Message Broker	Stores and delivers events to consumers (e.g. RabbitMQ, Kafka, SNS/SQS).
Payment Service	Processes payment and publishes an event.
Inventory Service	Reserves stock and publishes an event.
Notification Worker	Sends notifications (email/SMS).
Retry Mechanism	Retries failed messages with exponential backoff.
Dead-Letter Queue (DLQ)	Stores messages that keep failing for manual review.
Databases	Each service owns its own database.

Key Takeaways

1. Break large systems into independent services.
2. Use an API gateway for routing, security and observability.
3. Each service owns its own data.
4. Use asynchronous events for long-running tasks.
5. Design for failure with retries, DLQ and idempotency.
6. Monitor services, events and end-to-end flows.

Common Issues

1. Service timeouts and slow responses.
2. Too many retries causing additional load.
3. Duplicate processing of the same event.
4. Lost or delayed events.
5. Inconsistent data across services (partial failures).
6. Growing message backlog and stalled workers.

Best Practices

1. Set appropriate timeouts and retry policies.
2. Use idempotency keys to prevent duplicate processing.
3. Choose the right delivery guarantee for your use case (at-least-once, exactly-once).
4. Monitor queue length, worker health and failure rates.
5. Use a dead-letter queue for failed messages.
6. Implement tracing and correlation IDs for debugging.

Practice This

1. Draw both architectures and label each component.
2. Show the flow of a client request through the API gateway.
3. Show the flow of an order event from creation to completion.
4. Identify what happens if a service times out or fails.
5. Explain how you would prevent duplicate job execution.
6. List the metrics you would monitor for both systems.

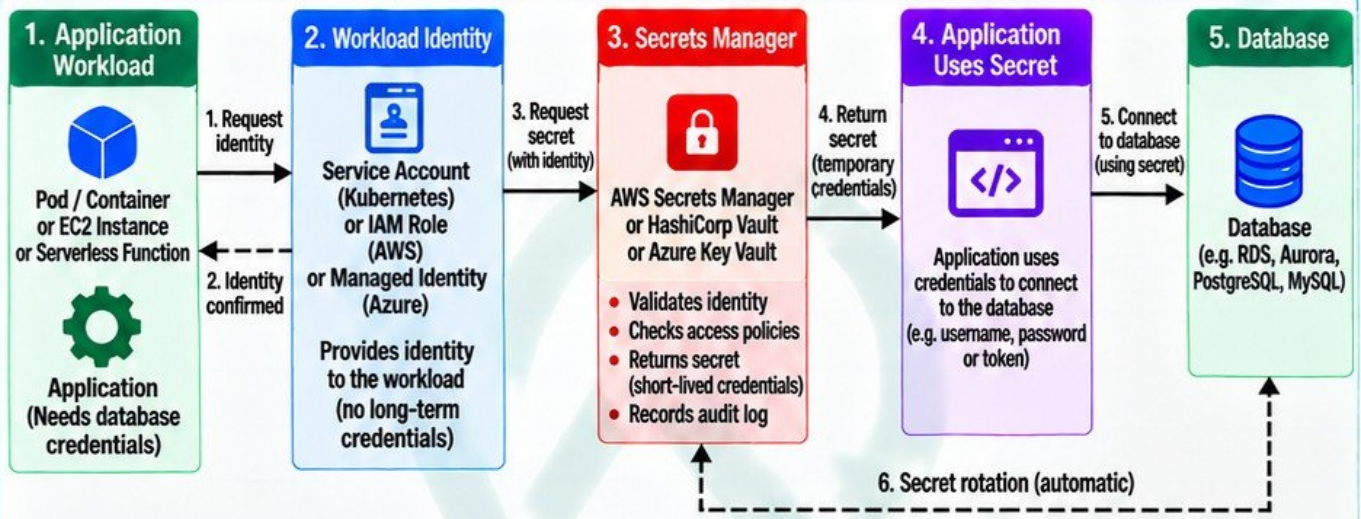
Secrets Management and Workload Identity Architecture

16

VERIQTA

This page shows how an application workload uses a workload identity to securely retrieve secrets from a secrets manager. It covers short-lived credentials, access policies, audit logging, and secret rotation, and highlights common mistakes to avoid.

System 27: Using Workload Identity to Access Secrets



Secret Rotation Flow

1. Create new version of secret
2. Update application to use new secret
3. Revoke old secret after grace period

Short-Lived Credentials

- Temporary credentials with a limited time-to-live (TTL).
- Automatically expire and are renewed when needed.
- Reduce the risk of credential leakage and misuse.
- Used for workloads, CI/CD pipelines, and cross-service access.

Access Policies (Least Privilege)

- Grant only the permissions required for the workload.
- Use resource-level policies (specific secrets).
- Separate roles for development, staging and production.
- Restrict access by identity, namespace, service account, or IAM role.
- Regularly review and remove unused access.

Audit Logging

- Records who accessed which secret, when and from where.
- Helps detect unauthorized access and suspicious activity.
- Integrates with monitoring and security tools (e.g. CloudTrail, Azure Monitor, Vault audit logs).
- Keep logs for compliance and incident investigation.

Key Components

Component	Purpose
Workload Identity	Identity for the application (e.g. service account or IAM role).
Secrets Manager	Stores, secures and returns secrets, with rotation and auditing.
Access Policies	Control which identities can access which secrets.
Short-Lived Credentials	Temporary credentials with limited lifetime.
Audit Logs	Track and record secret access and changes.
Database	Uses the retrieved credentials for authentication.

Common Mistakes and Why They Are Dangerous

Mistake	Why It Is Dangerous
Committing credentials to Git	Anyone with access to the repository can see and use the credentials.
Baking credentials into images	Credentials remain in the image layers and can be extracted.
Exposing credentials in pipeline logs	Logs are stored and can be accessed by many people and systems.
Using long-lived credentials	Higher risk of credential leakage and misuse if not rotated.
Overly permissive access policies	Increases the blast radius if a workload is compromised.

Key Takeaways

1. Use workload identity instead of hard-coded credentials.
2. Store secrets in a dedicated secrets manager.
3. Use short-lived credentials and rotate secrets regularly.
4. Apply least privillege access policies.
5. Enable and monitor audit logging.
6. Never commit credentials to Git, include them in images, or expose them in pipeline logs.

Practice This

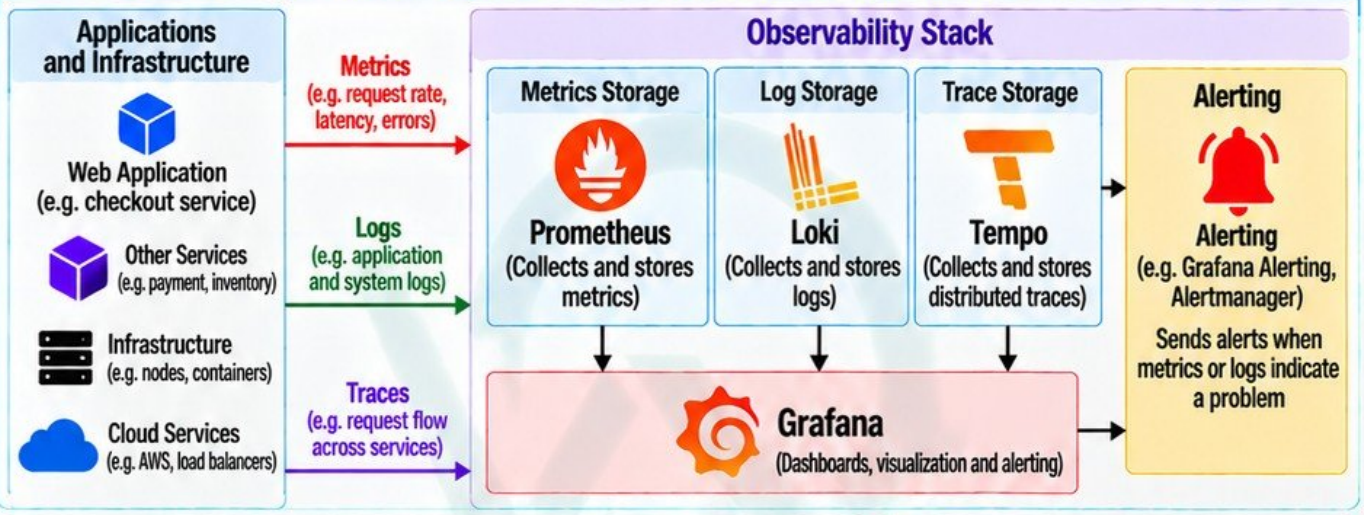
1. Draw and label the full architecture and data flow.
2. Explain how the workload identity is configured (e.g. Kubernetes service account or IAM role).
3. Show the steps to retrieve a secret from a secrets manager.
4. Explain how secret rotation works and how the application uses the new secret.
5. Identify what could happen if credentials are committed to Git or exposed in logs.

Production Observability: Metrics, Logs, and Traces

VERIQTA

This page shows how application and infrastructure telemetry (metrics, logs, and traces) is collected, stored, visualized and used for alerting and troubleshooting. It uses a common stack: Prometheus, Grafana, Loki, and Tempo, and shows how to follow a slow checkout request from an alert to a trace, a failing dependency, and the relevant application logs.

System 28: Observability Architecture with Prometheus, Grafana, Loki and Tempo



Example Flow: Investigating a Slow Checkout Request

1. **Alert Triggered** — High request latency for checkout service in Grafana
2. **Check Metrics** — Identify increased latency and error rate in Prometheus/Grafana
3. **Explore Traces** — Open trace in Tempo to see the full request flow across services
4. **Find Failing Dependency** — Trace shows slow response from payment service (external API)
5. **View Logs** — Check application logs in Loki for errors and timeouts

Key Components

Component	Purpose
Prometheus	Collects and stores metrics (from applications and infrastructure)
Grafana	Dashboards, visualization and alerting
Loki	Collects and stores logs
Tempo	Collects and stores distributed traces
Alertmanager	Handles alerts and notifications
Exporters/Agents	Collect metrics, logs and traces
Dashboards	Visualize system and application health

What Each Tool Collects

Tool	Data Collected	Examples
Prometheus	Metrics (time series)	Request rate, latency, error rate, CPU, memory, disk, etc.
Loki	Logs (text)	Application logs, container logs, system logs, error logs
Tempo	Traces (distributed)	Request flow, service calls, latency per service, errors
Grafana	Visualization	Dashboards, graphs, alerts
Alertmanager	Alert routing	Send alerts to email, Slack, etc.

Common Issues and How to Investigate

Issue	Possible Cause	What to Check
High latency	Slow database, external API, or resource saturation	Check metrics, traces and logs for slow spans and errors
Increased errors	Service failure or misconfiguration	Check error rate metrics and application logs
Missing traces	Instrumentation not configured	Verify tracing is enabled in the application and propagated
Logs not showing	Incorrect log configuration or agent issue	Check log agent, labels and log pipeline
Alert not firing	Wrong threshold or query	Check alert rule, query and alertmanager configuration

Key Takeaways

1. Use metrics, logs and traces together to get the full picture.
2. Metrics show that there is a problem.
3. Traces show where the problem is.
4. Logs explain why the problem is happening.
5. Good observability reduces mean time to resolution (MTTR).

Best Practices

1. Instrument your applications for metrics, logs and traces.
2. Use consistent labels and correlation IDs across logs and traces.
3. Create useful dashboards for key services.
4. Set meaningful alerts (not too noisy).
5. Retain logs and traces for an appropriate time period.

Practice This

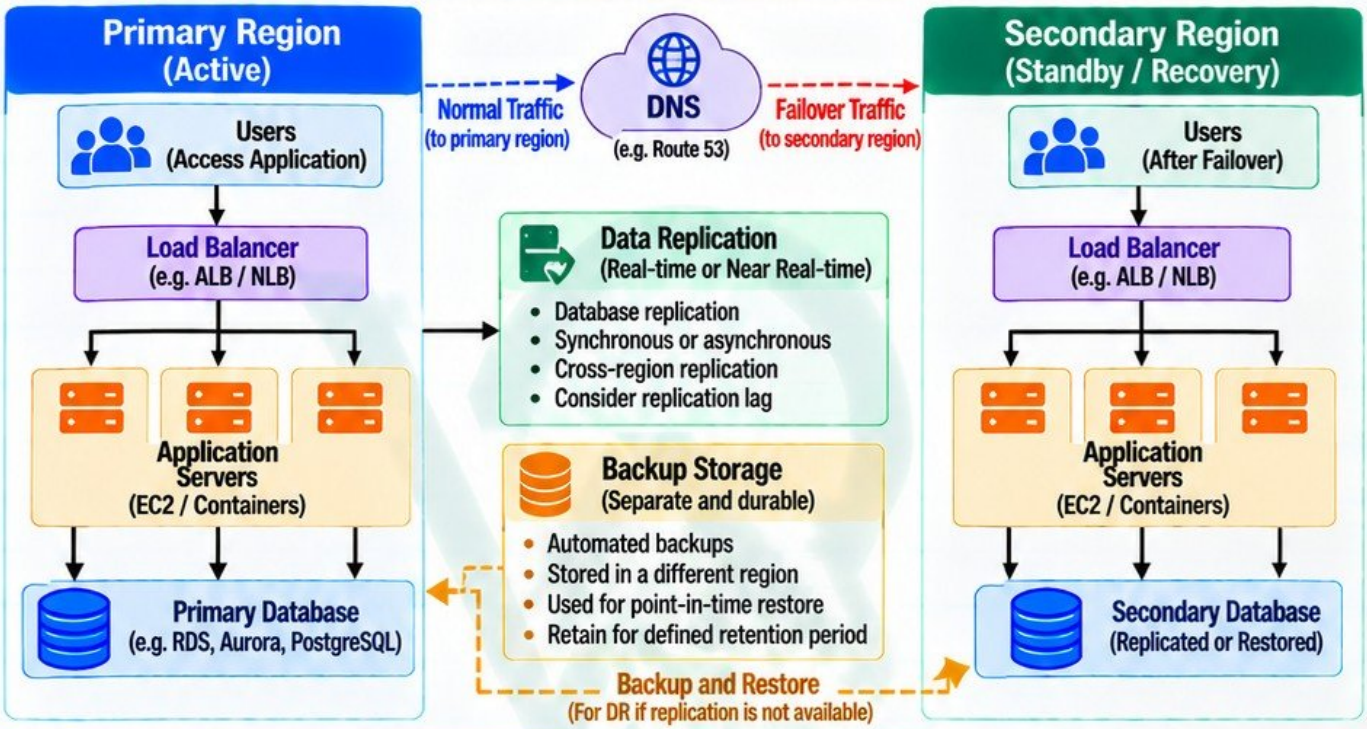
1. Draw the full observability architecture and label each component.
2. Show the flow of metrics, logs and traces from the application to storage and Grafana.
3. Simulate a slow checkout request and trace it end to end.
4. Identify the failing dependency and find the relevant logs.
5. Explain what alerts you would set for this system and why.

Disaster Recovery Across Two Cloud Regions

18

VERIQTA

This page shows a typical disaster recovery (DR) architecture across two cloud regions. It covers primary and secondary regions, data replication or backup restoration, traffic failover, and key considerations such as RTO, RPO, and testing.



Recovery Time Objective (RTO)

The maximum acceptable time to restore services after a disaster.

- Includes detection, failover, and service recovery.
- Depends on architecture, automation and testing.
- Shorter RTO usually costs more.

Recovery Point Objective (RPO)

The maximum acceptable amount of data that can be lost.

- Depends on replication method or backup frequency.
- Real-time replication gives a low RPO.
- If using daily backups, RPO could be 24 hours.

Key Components

Component	Purpose
Primary Region	Runs the production workload.
Secondary Region	Standby environment for failover.
Data Replication	Keeps data in sync across regions.
Backup Storage	Stores backups for point-in-time restore.
DNS (e.g. Route 53)	Routes traffic to healthy region.
Load Balancer	Distributes traffic to application servers.
Databases	Primary and secondary (replicated or restored).

Common Issues and How to Investigate

Issue	What to Check
DNS failover takes time	DNS TTL, caching, and client behavior.
Replication lag	Check replication status and lag metrics.
Data inconsistency	Verify RPO and last successful replication.
Dependencies not available	Check if all services are deployed in the secondary region (e.g. caches, queues).
Failover does not work	Test regularly and verify runbooks and automation.
Backup restore fails	Verify backup integrity and test restoration process.

Best Practices

1. Define and document RTO and RPO.
2. Use multi-region architecture for critical workloads.
3. Automate backups and replication.
4. Test failover and restore regularly.
5. Monitor replication lag and backup success.
6. Keep runbooks updated.
7. Consider all dependencies (databases, caches, queues, external services).

Key Takeaways

1. Disaster recovery requires both people, process and technology.
2. RTO and RPO define how quickly you recover and how much data you can lose.
3. DNS failover is not instant and has limitations.
4. Replication can have delays, and backups are essential.
5. Regular testing is critical to ensure recovery actually works.

Practice This

1. Draw the full two-region architecture and label each component.
2. Explain the flow of traffic during normal operation and after a failover.
3. Show how data is replicated and how backups are used for restore.
4. List the steps you would take to test a failover.
5. Identify the key metrics you would monitor for DR readiness.

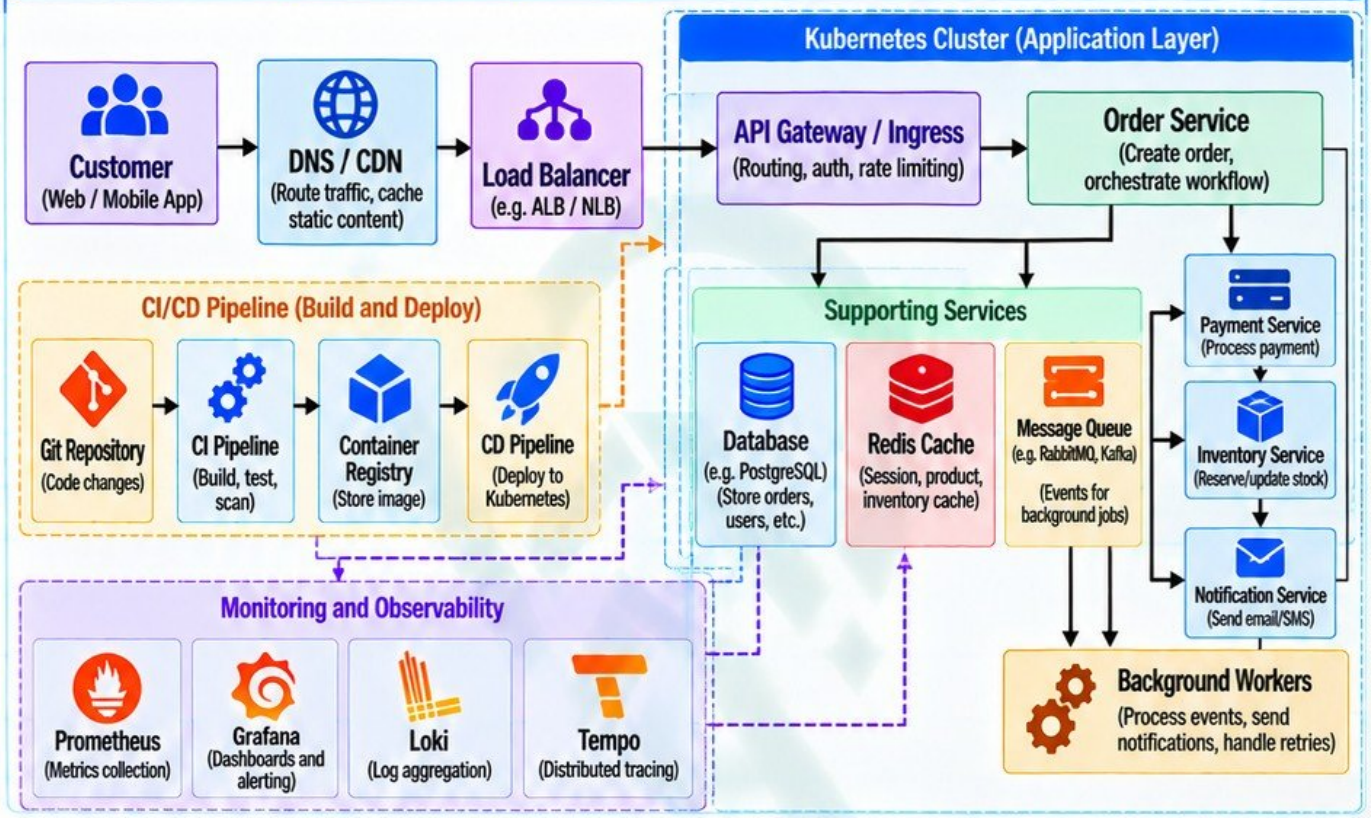
End-to-End Production E-Commerce Architecture

19

VERIQTA

This page shows a complete architecture for an online store that must handle traffic spikes, payments, inventory updates, and service failures. It includes the main request flow, supporting services, CI/CD and monitoring, and explains what happens when payment succeeds but inventory reservation fails.

System 30: Complete E-Commerce Architecture



Order Flow (Simplified)

1. Customer places an order on the website or app.
2. Request goes through DNS/CDN and the load balancer to the Kubernetes API.
3. Order Service validates the request and calls the Payment Service.
4. If payment succeeds, Order Service creates an order record and publishes an event.
5. Inventory Service reserves the product and updates stock.
6. Order information is stored in the database and cached in Redis.
7. A message is sent to the queue for background workers (e.g. send confirmation email).
8. Monitoring collects metrics, logs and traces across all services.

What Happens if Payment Succeeds but Inventory Reservation Fails?

1. Payment Service confirms the payment as successful.
2. Order Service tries to reserve inventory but the Inventory Service fails (e.g. out of stock or service error).
3. Order remains in a pending or failed state (depending on design).
4. A compensating action is triggered (e.g. refund the payment or mark for manual review).
5. An event can be sent to a queue for retry, or handled by a background worker.
6. The customer is notified about the failure and the refund (if applicable).
7. Logs, metrics and traces show the exact step where the failure happened.

Key Components

Component	Purpose
DNS / CDN	Route traffic and cache static content.
Load Balancer	Distribute traffic to Kubernetes.
Kubernetes API	Route requests to services.
Order Service	Handles order creation and workflow.
Payment Service	Processes payments.
Inventory Service	Reserves and updates stock.
Database	Stores orders, users and data.
Redis Cache	Caches session, product and inventory data.
Message Queue	Handles asynchronous events.
Background Workers	Process events (emails, retries, etc.).

Common Issues

Issue	Explanation
Service failure	A service is down or slow.
Payment succeeded, inventory failed	Requires compensation or refund.
Duplicate orders	Retries create multiple orders.
Out of stock	High demand or incorrect stock.
Message queue delay	Slow processing of events.
Data inconsistency	Order and inventory out of sync.
Slow response	Database, cache or external service issue.
Monitoring gaps	Missing logs, metrics or traces.

Best Practices

1. Use idempotency for order creation and payments.
2. Implement retries with backoff for transient failures.
3. Use a message queue for asynchronous processing.
4. Design compensating actions for failures.
5. Monitor key metrics, logs and traces.
6. Perform regular failure and load testing.

Practice This

1. Draw the full architecture and label each component.
2. Trace an order from the customer request to completion.
3. Show what happens when payment succeeds but inventory reservation fails.
4. Identify the logs, metrics and traces you would check.
5. Explain how you would design the compensation flow.

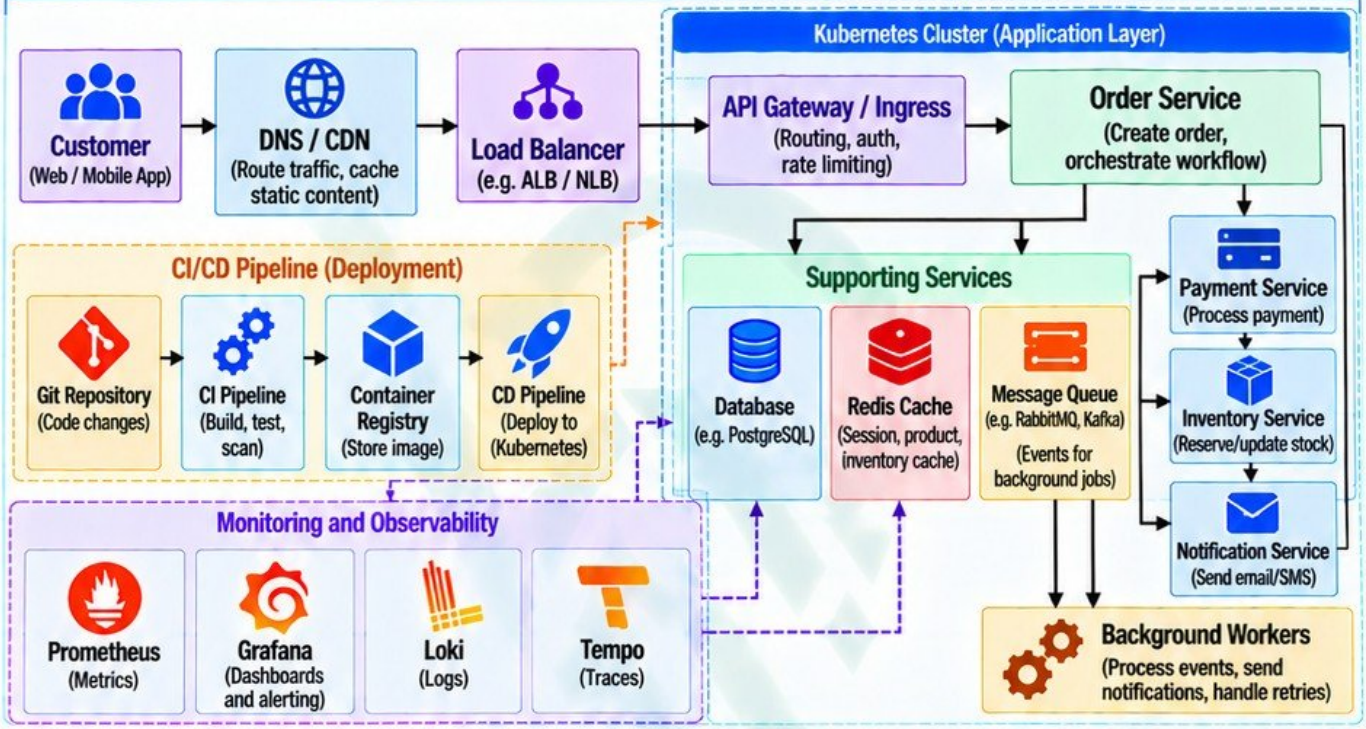
Production Failure Investigation: Find the Broken Dependency

20

VERIQTA

This is a final architecture-based troubleshooting exercise using System 30. Customers receive intermittent HTTP 502 errors after a deployment. Some orders are processed twice, while background jobs accumulate. The goal is to identify the root cause(s), investigate using the right telemetry and commands, and apply safe mitigations and rollback options.

System 30: E-Commerce Architecture (Reference)



Incident Summary

- After a deployment, customers receive intermittent HTTP 502 errors.
- Some orders are processed twice (duplicate orders).
- Background jobs are accumulating in the queue.
- The system was working before the deployment.

What You Need to Identify

1. The request path and affected components.
2. Which telemetry and diagnostic commands to check.
3. Why the failures may have different causes.
4. Safe mitigation and rollback options.
5. How to verify service recovery without creating more duplicate orders.

Possible Root Causes

1. A new version causing errors in the order service.
2. Payment service timeouts or failures.
3. Inventory service failures or slow responses.
4. Database or cache performance issues.
5. Message queue or worker failures.
6. Misconfiguration in the load balancer or ingress.
7. Resource exhaustion (CPU, memory, connection limits).
8. Partial deployment (some pods running old version).

Investigate the Request Path

1. Check load balancer / ingress for 502 errors.
2. Identify which service is returning the error.
3. Trace a single failing request through logs and traces.
4. Check downstream dependencies (payment, inventory, database, queue).
5. Compare working vs failing requests.

Useful Telemetry and Commands

Area	What to Check (Examples)
Kubernetes	kubectl get pods, kubectl logs, kubectl describe, kubectl top
Ingress / ALB	Access logs, 502 errors, target health
Application Logs	Search for errors, timeouts, exceptions
Traces (Tempo)	Follow request across services
Metrics (Prometheus)	Error rate, latency, resource usage
Queue (RabbitMQ/Kafka)	Queue length, failed messages
Database	Slow queries, connection errors
Deployment	Check new version, rollout status, recent changes

Mitigation and Rollback Options

1. Roll back to the previous stable version (if confirmed as the cause).
2. Scale up affected services (if resource related).
3. Pause or throttle background workers (to prevent duplicates).
4. Use a feature flag to disable the new feature.
5. Fix configuration and redeploy.
6. Drain traffic or fail over if a region or service is unhealthy.

Verify Recovery

1. Confirm 502 errors have stopped.
2. Monitor error rate and latency.
3. Check that new orders are processed only once.
4. Ensure background job queue is being consumed.
5. Validate payment, inventory and notifications are working.
6. Keep monitoring for at least 30 minutes.

Key Takeaways

1. One incident can have multiple contributing causes.
2. Always use telemetry (metrics, logs and traces) instead of guessing.
3. Understand the full request path and dependencies.
4. Make safe changes and be ready to roll back.
5. Verify recovery without creating more duplicate orders.
6. Practice this type of investigation regularly.

Practice This

1. Given the incident, identify the most likely root cause(s).
2. List the exact telemetry and commands you would check.
3. Explain your mitigation and rollback plan.
4. Show how you would verify the system is healthy again.
5. Explain how you would prevent this issue from happening in the future.